DATABASE MANAGEMENT SYSTEMS · ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

Retrieving and manipulating FoodExpress data with WHERE, ORDER BY, GROUP BY, HAVING and aggregates

Project	FoodExpress – Online Food Delivery System	Submitted by	Dhruvan Karthik
DBMS	MySQL Community Server 8.0.46	GitHub	github.com/hidhruvan-prog
Database	`foodexpress`	Date	25 October 2026

Introduction

This assignment queries the `foodexpress` database populated in Assignment 7 (20 customers, 10 restaurants, 52 dishes, 64 orders, 152 order lines, 64 payments and 34 reviews). Each query is shown with its SQL, the actual console output from MySQL 8.0.46, and a short explanation. The queries in this assignment mostly use one table at a time; joins and subqueries follow in Assignment 9. All statements are saved in `sql/05_basic_queries.sql`.

Clause / function	Queries
WHERE (=, AND, BETWEEN, IN, LIKE, IS NULL, NOT IN)	Q1 – Q6, Q14, Q21, Q23 – Q25
ORDER BY (ASC / DESC, multi-column, LIMIT)	Q7 – Q9, Q15 – Q19, Q21, Q22
COUNT(), SUM(), AVG(), MIN(), MAX()	Q10 – Q14, and inside Q15 – Q22
GROUP BY	Q15 – Q22
HAVING	Q19 – Q22
INSERT / UPDATE / DELETE	Q23 – Q25

1. Filtering Records with WHERE

WHERE keeps only the rows for which a condition is true. Conditions can use comparison operators, AND / OR / NOT, BETWEEN, IN, LIKE and IS NULL.

Q1 Vegetarian dishes under ₹150 that can be ordered now

```sql
SELECT item_id, item_name, price, category
FROM menu_items
WHERE is_veg = TRUE
  AND is_available = TRUE
  AND price < 150;
```

mysql — foodexpress

```
mysql> SELECT item_id, item_name, price, category
    -> FROM menu_items
    -> WHERE is_veg = TRUE
    ->   AND is_available = TRUE
    ->   AND price < 150;
+---------+--------------------------+--------+-------------+
| item_id | item_name                | price  | category    |
+---------+--------------------------+--------+-------------+
|       5 | Mirchi ka Salan          |  99.00 | Main Course |
|       6 | Double ka Meetha         | 119.00 | Dessert     |
|       7 | Masala Dosa              |  99.00 | Main Course |
|       8 | Ghee Roast Dosa          | 129.00 | Main Course |
|       9 | Idli Vada Combo          |  89.00 | Snack       |
|      10 | Rava Kesari              |  69.00 | Dessert     |
|      11 | Filter Coffee            |  49.00 | Beverage    |
|      21 | Garlic Bread             | 149.00 | Bread       |
|      23 | Pani Puri                |  60.00 | Snack       |
|      24 | Sev Puri                 |  80.00 | Snack       |
|      25 | Pav Bhaji                | 140.00 | Main Course |
|      26 | Vada Pav                 |  40.00 | Snack       |
|      27 | Masala Chaas             |  45.00 | Beverage    |
|      31 | Neer Dosa                |  99.00 | Bread       |
|      32 | Steamed Rice             |  89.00 | Rice        |
|      33 | Sol Kadhi                |  79.00 | Beverage    |
|      37 | Qubani ka Meetha         | 129.00 | Dessert     |
|      38 | Irani Chai               |  39.00 | Beverage    |
|      42 | Cold-Pressed Orange Juice| 149.00 | Beverage    |
|      46 | Butter Naan              |  55.00 | Bread       |
|      47 | Jeera Rice               | 149.00 | Rice        |
|      48 | Sweet Lassi              |  89.00 | Beverage    |
|      50 | Red Velvet Pastry        | 149.00 | Dessert     |
|      51 | Gulab Jamun (2 pcs)      |  79.00 | Dessert     |
|      52 | Cold Coffee              | 129.00 | Beverage    |
+---------+--------------------------+--------+-------------+
25 rows in set (0.00 sec)
```

Explanation. Three conditions combined with AND: only rows that are vegetarian, currently available and cheaper than ₹150 are returned – the list behind a "Veg under ₹150" filter in the app.

Q2 Cancelled orders

```sql
SELECT order_id, customer_id, restaurant_id, order_date
FROM orders
WHERE order_status = 'Cancelled';
```

mysql — foodexpress

```
mysql> SELECT order_id, customer_id, restaurant_id, order_date
    -> FROM orders
    -> WHERE order_status = 'Cancelled';
+----------+-------------+---------------+---------------------+
| order_id | customer_id | restaurant_id | order_date          |
+----------+-------------+---------------+---------------------+
|       15 |           1 |             1 | 2026-09-20 13:36:00 |
|       24 |           9 |             4 | 2026-09-26 20:01:00 |
|       34 |           1 |             2 | 2026-10-01 20:00:00 |
|       36 |           2 |             1 | 2026-10-03 22:38:00 |
|       41 |           3 |             3 | 2026-10-06 22:14:00 |
|       52 |          17 |             9 | 2026-10-14 22:33:00 |
|       54 |          15 |             8 | 2026-10-17 22:43:00 |
|       59 |          19 |             9 | 2026-10-22 13:48:00 |
+----------+-------------+---------------+---------------------+
8 rows in set (0.00 sec)
```

Explanation. Equality test on the ENUM column `order_status`. Support staff use this list to follow up refunds.

Q3 Orders placed in the first week of October (BETWEEN)

```sql
SELECT order_id, customer_id, order_date, order_status
FROM orders
WHERE order_date BETWEEN '2026-10-01 00:00:00' AND '2026-10-07 23:59:59';
```

mysql — foodexpress

```
mysql> SELECT order_id, customer_id, order_date, order_status
    -> FROM orders
    -> WHERE order_date BETWEEN '2026-10-01 00:00:00' AND '2026-10-07 23:59:59';
+----------+-------------+---------------------+--------------+
| order_id | customer_id | order_date          | order_status |
+----------+-------------+---------------------+--------------+
|       33 |          19 | 2026-10-01 19:31:00 | Delivered    |
|       34 |           1 | 2026-10-01 20:00:00 | Cancelled    |
|       35 |           7 | 2026-10-01 20:21:00 | Delivered    |
|       36 |           2 | 2026-10-03 22:38:00 | Cancelled    |
|       37 |           4 | 2026-10-05 19:14:00 | Delivered    |
|       38 |           2 | 2026-10-05 19:30:00 | Delivered    |
|       39 |           4 | 2026-10-06 14:34:00 | Delivered    |
|       40 |           3 | 2026-10-06 20:43:00 | Delivered    |
|       41 |           3 | 2026-10-06 22:14:00 | Cancelled    |
+----------+-------------+---------------------+--------------+
9 rows in set (0.00 sec)
```

Explanation. BETWEEN is inclusive at both ends, so the upper bound is written as the last second of 7 October to include the whole day.

Q4 Restaurants in Mumbai or Pune (IN)

```sql
SELECT restaurant_id, restaurant_name, cuisine_type, city
FROM restaurants
WHERE city IN ('Mumbai', 'Pune');
```

mysql — foodexpress

```
mysql> SELECT restaurant_id, restaurant_name, cuisine_type, city
    -> FROM restaurants
    -> WHERE city IN ('Mumbai', 'Pune');
+---------------+---------------------+--------------+--------+
| restaurant_id | restaurant_name     | cuisine_type | city   |
+---------------+---------------------+--------------+--------+
|             4 | Napoli Slice        | Italian      | Mumbai |
|             5 | Bombay Chaat Corner | Street Food  | Mumbai |
|             6 | Coastal Curry Co.   | Seafood      | Mumbai |
|             9 | Punjabi Tadka       | North Indian | Pune   |
|            10 | Sweet Tooth Desserts| Desserts     | Pune   |
+---------------+---------------------+--------------+--------+
5 rows in set (0.00 sec)
```

Explanation. IN is a shorter way of writing `city = 'Mumbai' OR city = 'Pune'`.

Q5 Every biryani on the platform (LIKE pattern)

```sql
SELECT item_id, restaurant_id, item_name, price
FROM menu_items
WHERE item_name LIKE '%Biryani%';
```

mysql — foodexpress

```
mysql> SELECT item_id, restaurant_id, item_name, price
    -> FROM menu_items
    -> WHERE item_name LIKE '%Biryani%';
+---------+---------------+--------------------------+--------+
| item_id | restaurant_id | item_name                | price  |
+---------+---------------+--------------------------+--------+
|       1 |             1 | Chicken Dum Biryani      | 289.00 |
|       2 |             1 | Mutton Dum Biryani       | 379.00 |
|       3 |             1 | Veg Dum Biryani          | 229.00 |
|      34 |             7 | Hyderabadi Chicken Biryani | 299.00 |
+---------+---------------+--------------------------+--------+
4 rows in set (0.00 sec)
```

Explanation. `%` matches any sequence of characters, so this finds "Biryani" anywhere in the dish name – a simple search box.

Q6 Active orders still waiting for a rider (IS NULL)

```sql
SELECT order_id, restaurant_id, order_date, order_status
FROM orders
WHERE agent_id IS NULL
  AND order_status NOT IN ('Delivered', 'Cancelled');
```

mysql — foodexpress

```
mysql> SELECT order_id, restaurant_id, order_date, order_status
    -> FROM orders
    -> WHERE agent_id IS NULL
    ->   AND order_status NOT IN ('Delivered', 'Cancelled');
+----------+---------------+---------------------+--------------+
| order_id | restaurant_id | order_date          | order_status |
+----------+---------------+---------------------+--------------+
|       63 |             6 | 2026-10-24 20:30:00 | Preparing    |
|       64 |             9 | 2026-10-24 20:48:00 | Placed       |
+----------+---------------+---------------------+--------------+
2 rows in set (0.00 sec)
```

Explanation. NULL cannot be compared with `=`; `IS NULL` is required. Combined with NOT IN it gives the dispatch queue of live orders without a rider.

2. Sorting Records with ORDER BY

ORDER BY sorts the result ascending (ASC, the default) or descending (DESC). Several sort keys can be given; LIMIT then keeps the first N rows.

Q7 The ten most expensive dishes

```sql
SELECT item_name, restaurant_id, category, price
FROM menu_items
ORDER BY price DESC, item_name ASC
LIMIT 10;
```

mysql — foodexpress

```
mysql> SELECT item_name, restaurant_id, category, price
    -> FROM menu_items
    -> ORDER BY price DESC, item_name ASC
    -> LIMIT 10;
+--------------------------+---------------+-------------+--------+
| item_name                | restaurant_id | category    | price  |
+--------------------------+---------------+-------------+--------+
| Pepperoni Pizza          |             4 | Main Course | 449.00 |
| Surmai Fry               |             6 | Starter     | 449.00 |
| Malvani Fish Curry       |             6 | Main Course | 429.00 |
| Prawn Koliwada           |             6 | Starter     | 399.00 |
| Mutton Dum Biryani       |             1 | Rice        | 379.00 |
| Margherita Pizza         |             4 | Main Course | 349.00 |
| Butter Chicken           |             9 | Main Course | 329.00 |
| Grilled Chicken Salad    |             8 | Main Course | 329.00 |
| Penne Arrabbiata         |             4 | Main Course | 329.00 |
| Hyderabadi Chicken Biryani |           7 | Rice        | 299.00 |
+--------------------------+---------------+-------------+--------+
10 rows in set (0.00 sec)
```

Explanation. Sorted by price from highest to lowest; dishes with the same price are ordered alphabetically by the second key.

Q8 Restaurant directory sorted by city, then name

```sql
SELECT city, restaurant_name, cuisine_type, contact_phone
FROM restaurants
ORDER BY city, restaurant_name;
```

mysql — foodexpress

```
mysql> SELECT city, restaurant_name, cuisine_type, contact_phone
    -> FROM restaurants
    -> ORDER BY city, restaurant_name;
+-----------+--------------------------+--------------+---------------+
| city      | restaurant_name          | cuisine_type | contact_phone |
+-----------+--------------------------+--------------+---------------+
| Bengaluru | Dosa Junction            | South Indian | 08039761584   |
| Bengaluru | Dragon Wok               | Chinese      | 08064506045   |
| Bengaluru | Spice Route Biryani House | Hyderabadi  | 08057990659   |
| Hyderabad | Green Bowl Salads        | Healthy      | 04063228088   |
| Hyderabad | Nizam's Kitchen          | Hyderabadi   | 04069256663   |
| Mumbai    | Bombay Chaat Corner      | Street Food  | 02253801291   |
| Mumbai    | Coastal Curry Co.        | Seafood      | 02210285940   |
| Mumbai    | Napoli Slice             | Italian      | 02241465908   |
| Pune      | Punjabi Tadka            | North Indian | 02064230048   |
| Pune      | Sweet Tooth Desserts     | Desserts     | 02076452912   |
+-----------+--------------------------+--------------+---------------+
10 rows in set (0.00 sec)
```

Explanation. A multi-column sort: rows are grouped visually by city, and alphabetical within each city.

Q9 The eight most recent orders

```sql
SELECT order_id, customer_id, restaurant_id, order_date, order_status
FROM orders
ORDER BY order_date DESC
LIMIT 8;
```

mysql — foodexpress

```
mysql> SELECT order_id, customer_id, restaurant_id, order_date, order_status
    -> FROM orders
    -> ORDER BY order_date DESC
    -> LIMIT 8;
+----------+-------------+---------------+---------------------+------------------+
| order_id | customer_id | restaurant_id | order_date          | order_status     |
+----------+-------------+---------------+---------------------+------------------+
|       64 |          17 |             9 | 2026-10-24 20:48:00 | Placed           |
|       63 |           8 |             6 | 2026-10-24 20:30:00 | Preparing        |
|       62 |          11 |             5 | 2026-10-24 20:10:00 | Preparing        |
|       61 |          10 |             4 | 2026-10-24 19:25:00 | Out for Delivery |
|       60 |           4 |             3 | 2026-10-24 17:00:00 | Delivered        |
|       59 |          19 |             9 | 2026-10-22 13:48:00 | Cancelled        |
|       58 |          17 |             9 | 2026-10-20 21:54:00 | Delivered        |
|       57 |          11 |             4 | 2026-10-20 13:17:00 | Delivered        |
+----------+-------------+---------------+---------------------+------------------+
8 rows in set (0.00 sec)
```

Explanation. Newest first – the live-orders screen of the operations dashboard.

3. Aggregate Functions

Aggregate functions collapse many rows into one value. COUNT(*) counts rows, COUNT(column) counts non-NULL values, and SUM, AVG, MIN and MAX ignore NULLs.

Q10 COUNT() – orders, distinct customers and riders assigned

```sql
SELECT COUNT(*)                    AS total_orders,
       COUNT(DISTINCT customer_id) AS customers_who_ordered,
       COUNT(agent_id)             AS orders_with_rider,
       COUNT(delivered_at)         AS delivered_orders
FROM orders;
```

mysql — foodexpress

```
mysql> SELECT COUNT(*)                    AS total_orders,
    ->        COUNT(DISTINCT customer_id) AS customers_who_ordered,
    ->        COUNT(agent_id)             AS orders_with_rider,
    ->        COUNT(delivered_at)         AS delivered_orders
    -> FROM orders;
+--------------+-----------------------+-------------------+------------------+
| total_orders | customers_who_ordered | orders_with_rider | delivered_orders |
+--------------+-----------------------+-------------------+------------------+
|           64 |                    18 |                54 |               52 |
+--------------+-----------------------+-------------------+------------------+
1 row in set (0.00 sec)
```

Explanation. `COUNT(agent_id)` and `COUNT(delivered_at)` are smaller than `COUNT(*)` because those columns are NULL for orders without a rider or not yet delivered. `COUNT(DISTINCT …)` counts each customer once.

Q11 SUM() – money collected and refunded

```sql
SELECT SUM(CASE WHEN payment_status = 'Completed' THEN amount ELSE 0 END) AS collected,
       SUM(CASE WHEN payment_status = 'Refunded'  THEN amount ELSE 0 END) AS refunded,
       SUM(CASE WHEN payment_status = 'Pending'   THEN amount ELSE 0 END) AS pending
FROM payments;
```

```
mysql — foodexpress
mysql> SELECT SUM(CASE WHEN payment_status = 'Completed' THEN amount ELSE 0 END) AS collected,
    ->        SUM(CASE WHEN payment_status = 'Refunded'  THEN amount ELSE 0 END) AS refunded,
    ->        SUM(CASE WHEN payment_status = 'Pending'   THEN amount ELSE 0 END) AS pending
    -> FROM payments;
+-----------+----------+---------+
| collected | refunded | pending |
+-----------+----------+---------+
|  39294.00 |  2908.00 | 1353.00 |
+-----------+----------+---------+
1 row in set (0.00 sec)
```

Explanation. SUM with a CASE expression adds up only the rows of each payment status, giving three totals in one pass over the table.

Q12 AVG() – average menu price and average ratings

```sql
SELECT ROUND(AVG(price), 2)  AS avg_menu_price
FROM menu_items;

SELECT ROUND(AVG(food_rating), 2)  AS avg_food_rating,
       ROUND(AVG(agent_rating), 2) AS avg_rider_rating,
       COUNT(agent_rating)         AS rider_ratings_given
FROM reviews;
```

```
mysql — foodexpress
mysql> SELECT ROUND(AVG(price), 2)  AS avg_menu_price
    -> FROM menu_items;
+----------------+
| avg_menu_price |
+----------------+
|         192.96 |
+----------------+
1 row in set (0.00 sec)

mysql> SELECT ROUND(AVG(food_rating), 2)  AS avg_food_rating,
    ->        ROUND(AVG(agent_rating), 2) AS avg_rider_rating,
    ->        COUNT(agent_rating)         AS rider_ratings_given
    -> FROM reviews;
+-----------------+-----------------+--------------------+
| avg_food_rating | avg_rider_rating | rider_ratings_given |
+-----------------+-----------------+--------------------+
|            4.15 |            4.08 |                 24 |
+-----------------+-----------------+--------------------+
1 row in set (0.00 sec)
```

Explanation. AVG ignores NULLs, so the rider average is calculated only from reviews that included a rider rating (fewer than the total number of reviews).

Q13 MIN() and MAX() – price range and trading period

```sql
SELECT MIN(price) AS cheapest_item, MAX(price) AS costliest_item
FROM menu_items;

SELECT MIN(order_date) AS first_order, MAX(order_date) AS latest_order
FROM orders;
```

mysql — foodexpress

```
mysql> SELECT MIN(price) AS cheapest_item, MAX(price) AS costliest_item
    -> FROM menu_items;
+---------------+----------------+
| cheapest_item | costliest_item |
+---------------+----------------+
|         39.00 |         449.00 |
+---------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT MIN(order_date) AS first_order, MAX(order_date) AS latest_order
    -> FROM orders;
+---------------------+---------------------+
| first_order         | latest_order        |
+---------------------+---------------------+
| 2026-09-02 14:50:00 | 2026-10-24 20:48:00 |
+---------------------+---------------------+
1 row in set (0.00 sec)
```

Explanation. MIN and MAX work on numbers and also on dates (earliest and latest) and text (alphabetical).

Q14 All five aggregates together – completed payments

```sql
SELECT COUNT(*)              AS payments,
       SUM(amount)           AS revenue,
       ROUND(AVG(amount), 2) AS avg_order_value,
       MIN(amount)           AS smallest,
       MAX(amount)           AS largest
FROM payments
WHERE payment_status = 'Completed';
```

mysql — foodexpress

```
mysql> SELECT COUNT(*)              AS payments,
    ->        SUM(amount)           AS revenue,
    ->        ROUND(AVG(amount), 2) AS avg_order_value,
    ->        MIN(amount)           AS smallest,
    ->        MAX(amount)           AS largest
    -> FROM payments
    -> WHERE payment_status = 'Completed';
+----------+----------+-----------------+----------+---------+
| payments | revenue  | avg_order_value | smallest | largest |
+----------+----------+-----------------+----------+---------+
|       54 | 39294.00 |          727.67 |    70.00 | 2041.00 |
+----------+----------+-----------------+----------+---------+
1 row in set (0.00 sec)
```

Explanation. WHERE filters the rows *before* aggregation, so the figures describe successful payments only – the platform's key revenue numbers.

4. Grouping Records with GROUP BY

GROUP BY splits the rows into groups that share a value and computes the aggregates once per group. Every selected column must either be grouped or aggregated (ONLY_FULL_GROUP_BY).

Q15 Number of orders in each status

```sql
SELECT order_status, COUNT(*) AS orders
FROM orders
GROUP BY order_status
ORDER BY orders DESC;
```

mysql — foodexpress

```
mysql> SELECT order_status, COUNT(*) AS orders
    -> FROM orders
    -> GROUP BY order_status
    -> ORDER BY orders DESC;
+------------------+--------+
| order_status     | orders |
+------------------+--------+
| Delivered        |     52 |
| Cancelled        |      8 |
| Preparing        |      2 |
| Out for Delivery |      1 |
| Placed           |      1 |
+------------------+--------+
5 rows in set (0.00 sec)
```

Explanation. One row per status – the order funnel.

Q16 Revenue by payment method

```sql
SELECT payment_method,
       COUNT(*)              AS payments,
       SUM(amount)           AS revenue,
       ROUND(AVG(amount), 2) AS avg_amount
FROM payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY revenue DESC;
```

mysql — foodexpress

```
mysql> SELECT payment_method,
    ->        COUNT(*)              AS payments,
    ->        SUM(amount)           AS revenue,
    ->        ROUND(AVG(amount), 2) AS avg_amount
    -> FROM payments
    -> WHERE payment_status = 'Completed'
    -> GROUP BY payment_method
    -> ORDER BY revenue DESC;
+----------------+----------+----------+------------+
| payment_method | payments | revenue  | avg_amount |
+----------------+----------+----------+------------+
| UPI            |       25 | 17340.00 |     693.60 |
| Card           |       13 | 11482.00 |     883.23 |
| Wallet         |        8 |  5970.00 |     746.25 |
| Cash           |        8 |  4502.00 |     562.75 |
+----------------+----------+----------+------------+
4 rows in set (0.00 sec)
```

Explanation. Shows which payment methods customers prefer and how much money each brings in.

Q17 Menu size and price level per category

```sql
SELECT category,
       COUNT(*)              AS items,
       MIN(price)            AS min_price,
       MAX(price)            AS max_price,
       ROUND(AVG(price), 2) AS avg_price
FROM menu_items
GROUP BY category
ORDER BY avg_price DESC;
```

mysql — foodexpress

```
mysql> SELECT category,
    ->        COUNT(*)            AS items,
    ->        MIN(price)          AS min_price,
    ->        MAX(price)          AS max_price,
    ->        ROUND(AVG(price), 2) AS avg_price
    -> FROM menu_items
    -> GROUP BY category
    -> ORDER BY avg_price DESC;
+-------------+-------+-----------+-----------+-----------+
| category    | items | min_price | max_price | avg_price |
+-------------+-------+-----------+-----------+-----------+
| Starter     |     7 |    159.00 |    449.00 |    279.00 |
| Main Course |    16 |     99.00 |    449.00 |    262.19 |
| Rice        |     7 |     89.00 |    379.00 |    236.14 |
| Dessert     |     7 |     69.00 |    249.00 |    141.86 |
| Bread       |     3 |     55.00 |    149.00 |    101.00 |
| Beverage    |     8 |     39.00 |    149.00 |     83.50 |
| Snack       |     4 |     40.00 |     89.00 |     67.25 |
+-------------+-------+-----------+-----------+-----------+
7 rows in set (0.00 sec)
```

Explanation. Several aggregates per group: the price band of each menu category.

Q18 Orders per month

```sql
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(*)                         AS orders,
       SUM(order_status = 'Cancelled')  AS cancelled
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

mysql — foodexpress

```
mysql> SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
    ->        COUNT(*)                         AS orders,
    ->        SUM(order_status = 'Cancelled')  AS cancelled
    -> FROM orders
    -> GROUP BY DATE_FORMAT(order_date, '%Y-%m')
    -> ORDER BY month;
+---------+--------+-----------+
| month   | orders | cancelled |
+---------+--------+-----------+
| 2026-09 |     32 |         2 |
| 2026-10 |     32 |         6 |
+---------+--------+-----------+
2 rows in set (0.00 sec)
```

Explanation. Grouping by an expression. `SUM(condition)` counts the rows where the condition is true because MySQL treats TRUE as 1.

5. Filtering Groups with HAVING

HAVING filters groups *after* aggregation, so it can test aggregate values; WHERE cannot, because it runs before the groups exist.

Q19 Restaurants that received more than 8 orders

```sql
SELECT restaurant_id, COUNT(*) AS orders
FROM orders
GROUP BY restaurant_id
HAVING COUNT(*) > 8
ORDER BY orders DESC;
```

mysql — foodexpress

```
mysql> SELECT restaurant_id, COUNT(*) AS orders
    -> FROM orders
    -> GROUP BY restaurant_id
    -> HAVING COUNT(*) > 8
    -> ORDER BY orders DESC;
+---------------+--------+
| restaurant_id | orders |
+---------------+--------+
|             9 |     13 |
|             1 |     11 |
|             3 |     10 |
+---------------+--------+
3 rows in set (0.00 sec)
```

Explanation. Groups with 8 or fewer orders are removed by HAVING – the platform's busiest kitchens.

Q20 Menu categories whose average price is above ₹200

```sql
SELECT category, COUNT(*) AS items, ROUND(AVG(price), 2) AS avg_price
FROM menu_items
GROUP BY category
HAVING AVG(price) > 200;
```

mysql — foodexpress

```
mysql> SELECT category, COUNT(*) AS items, ROUND(AVG(price), 2) AS avg_price
    -> FROM menu_items
    -> GROUP BY category
    -> HAVING AVG(price) > 200;
+-------------+-------+-----------+
| category    | items | avg_price |
+-------------+-------+-----------+
| Rice        |     7 |    236.14 |
| Starter     |     7 |    279.00 |
| Main Course |    16 |    262.19 |
+-------------+-------+-----------+
3 rows in set (0.00 sec)
```

Explanation. The condition uses the aggregate AVG(price), which is only possible in HAVING.

Q21 Repeat customers (at least 4 delivered orders)

```sql
SELECT customer_id, COUNT(*) AS delivered_orders
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
HAVING COUNT(*) >= 4
ORDER BY delivered_orders DESC, customer_id;
```

mysql — foodexpress

```
mysql> SELECT customer_id, COUNT(*) AS delivered_orders
    -> FROM orders
    -> WHERE order_status = 'Delivered'
    -> GROUP BY customer_id
    -> HAVING COUNT(*) >= 4
    -> ORDER BY delivered_orders DESC, customer_id;
+-------------+-----------------+
| customer_id | delivered_orders |
+-------------+-----------------+
|          11 |               7 |
|           1 |               5 |
|           4 |               5 |
|           2 |               4 |
|          18 |               4 |
+-------------+-----------------+
5 rows in set (0.00 sec)
```

Explanation. WHERE and HAVING together: WHERE first keeps only delivered orders, GROUP BY counts them per customer, and HAVING keeps loyal customers – candidates for a loyalty offer.

Q22 Large orders: subtotal above ₹900

```sql
SELECT order_id,
       SUM(quantity)                  AS units,
       SUM(quantity * price_at_order) AS subtotal
FROM order_items
GROUP BY order_id
HAVING subtotal > 900
ORDER BY subtotal DESC;
```

mysql — foodexpress

```
mysql> SELECT order_id,
    ->        SUM(quantity)                  AS units,
    ->        SUM(quantity * price_at_order) AS subtotal
    -> FROM order_items
    -> GROUP BY order_id
    -> HAVING subtotal > 900
    -> ORDER BY subtotal DESC;
+----------+-------+----------+
| order_id | units | subtotal |
+----------+-------+----------+
|       57 |     9 |  2041.00 |
|       24 |     5 |  1905.00 |
|       61 |     5 |  1645.00 |
|       44 |     9 |  1609.00 |
|       45 |     6 |  1574.00 |
|       38 |     6 |  1464.00 |
|       20 |     4 |  1336.00 |
|       50 |     6 |  1324.00 |
|       43 |     7 |  1183.00 |
|       40 |     5 |  1155.00 |
|       32 |     4 |  1066.00 |
|       31 |     5 |  1025.00 |
|       18 |     5 |  1015.00 |
|        6 |     7 |   973.00 |
|       26 |     4 |   966.00 |
|       28 |     5 |   965.00 |
|       12 |     4 |   926.00 |
+----------+-------+----------+
17 rows in set (0.00 sec)
```

Explanation. The subtotal is computed per order from its lines; MySQL allows the HAVING clause to refer to the alias.

6. Manipulating Data (INSERT, UPDATE, DELETE)

Retrieval is complemented by Data Manipulation Language statements that change the data, again restricted with WHERE.

Q23 UPDATE – Dosa Junction raises the price of Masala Dosa

```sql
SELECT item_id, item_name, price FROM menu_items WHERE item_id = 7;

UPDATE menu_items
SET price = 109.00
WHERE item_id = 7;

SELECT item_id, item_name, price FROM menu_items WHERE item_id = 7;

SELECT order_id, item_id, quantity, price_at_order
FROM order_items
WHERE item_id = 7;
```

mysql — foodexpress

```
mysql> SELECT item_id, item_name, price FROM menu_items WHERE item_id = 7;
+---------+-------------+-------+
| item_id | item_name   | price |
+---------+-------------+-------+
|       7 | Masala Dosa | 99.00 |
+---------+-------------+-------+
1 row in set (0.00 sec)

mysql> UPDATE menu_items
    -> SET price = 109.00
    -> WHERE item_id = 7;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT item_id, item_name, price FROM menu_items WHERE item_id = 7;
+---------+-------------+--------+
| item_id | item_name   | price  |
+---------+-------------+--------+
|       7 | Masala Dosa | 109.00 |
+---------+-------------+--------+
1 row in set (0.00 sec)

mysql> SELECT order_id, item_id, quantity, price_at_order
    -> FROM order_items
    -> WHERE item_id = 7;
+----------+---------+----------+----------------+
| order_id | item_id | quantity | price_at_order |
+----------+---------+----------+----------------+
|        3 |       7 |        2 |          99.00 |
|       11 |       7 |        1 |          99.00 |
|       39 |       7 |        1 |          99.00 |
+----------+---------+----------+----------------+
3 rows in set (0.00 sec)
```

Explanation. The WHERE clause limits the update to one row. Past order lines still show ₹99 in `price_at_order`: the price snapshot kept in Assignment 3 protects the history of earlier orders.

Q24 UPDATE – the order that is out for delivery is delivered

```sql
UPDATE orders
SET order_status = 'Delivered',
    delivered_at = order_date + INTERVAL 31 MINUTE
WHERE order_status = 'Out for Delivery';

UPDATE payments
SET payment_status = 'Completed'
WHERE payment_status = 'Pending'
  AND order_id IN (SELECT order_id FROM orders WHERE order_status = 'Delivered');

SELECT o.order_id, o.order_status, o.delivered_at, p.payment_method, p.payment_status
FROM orders o JOIN payments p ON p.order_id = o.order_id
WHERE o.order_id = 61;
```

mysql — foodexpress

```
mysql> UPDATE orders
    -> SET order_status = 'Delivered',
    ->     delivered_at = order_date + INTERVAL 31 MINUTE
    -> WHERE order_status = 'Out for Delivery';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE payments
    -> SET payment_status = 'Completed'
    -> WHERE payment_status = 'Pending'
    ->   AND order_id IN (SELECT order_id FROM orders WHERE order_status = 'Delivered');
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> SELECT o.order_id, o.order_status, o.delivered_at, p.payment_method, p.payment_status
    -> FROM orders o JOIN payments p ON p.order_id = o.order_id
    -> WHERE o.order_id = 61;
+----------+--------------+---------------------+----------------+----------------+
| order_id | order_status | delivered_at        | payment_method | payment_status |
+----------+--------------+---------------------+----------------+----------------+
|       61 | Delivered    | 2026-10-24 19:56:00 | UPI            | Completed      |
+----------+--------------+---------------------+----------------+----------------+
1 row in set (0.00 sec)
```

Explanation. Both columns are set in one statement, so the CHECK rule "Delivered ⇔ delivered_at is set" is satisfied. The second UPDATE completes any pending cash payment of a delivered order; here it changes 0 rows because order 61 had already been paid by UPI.

Q25 INSERT and DELETE – a seasonal dish is added and later withdrawn

```sql
INSERT INTO menu_items (restaurant_id, item_name, description, price, category)
VALUES (10, 'Pumpkin Spice Cake', 'Autumn special', 179.00, 'Dessert');

SELECT item_id, item_name, price, is_veg, is_available
FROM menu_items WHERE restaurant_id = 10;

DELETE FROM menu_items
WHERE restaurant_id = 10 AND item_name = 'Pumpkin Spice Cake';

SELECT COUNT(*) AS sweet_tooth_items FROM menu_items WHERE restaurant_id = 10;
```

mysql — foodexpress

```
mysql> INSERT INTO menu_items (restaurant_id, item_name, description, price, category)
    -> VALUES (10, 'Pumpkin Spice Cake', 'Autumn special', 179.00, 'Dessert');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT item_id, item_name, price, is_veg, is_available
    -> FROM menu_items WHERE restaurant_id = 10;
+---------+--------------------------+--------+--------+--------------+
| item_id | item_name                | price  | is_veg | is_available |
+---------+--------------------------+--------+--------+--------------+
|      49 | Belgian Chocolate Waffle | 199.00 |      1 |            1 |
|      52 | Cold Coffee              | 129.00 |      1 |            1 |
|      51 | Gulab Jamun (2 pcs)      |  79.00 |      1 |            1 |
|      53 | Pumpkin Spice Cake       | 179.00 |      1 |            1 |
|      50 | Red Velvet Pastry        | 149.00 |      1 |            1 |
+---------+--------------------------+--------+--------+--------------+
5 rows in set (0.00 sec)

mysql> DELETE FROM menu_items
    -> WHERE restaurant_id = 10 AND item_name = 'Pumpkin Spice Cake';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT COUNT(*) AS sweet_tooth_items FROM menu_items WHERE restaurant_id = 10;
+-------------------+
| sweet_tooth_items |
+-------------------+
|                 4 |
+-------------------+
1 row in set (0.00 sec)
```

Explanation. The INSERT relies on the DEFAULT values for `is_veg` and `is_available`. The DELETE succeeds because the dish was never ordered; deleting a dish that appears in an order is blocked by the RESTRICT foreign key.

Conclusion

WHERE filters individual rows, ORDER BY arranges the result, the aggregate functions summarize many rows into single figures, GROUP BY computes those figures per category, and HAVING filters the groups by their aggregate values. Together they already answer the platform's everyday operational questions – live orders awaiting riders, revenue by payment method, loyal customers, price bands per category – and the DML statements show how the same WHERE logic is used safely to change data within the constraints defined earlier.